

HIDDEN MARKOV MODELS AND AI INTEGRATION FOR FORECASTING MODEL IN FINANCE

Internship Report

Author : Paul de Pazanan

github.com/PdePazanan/Internship_SNU

Supervisor : Pr. Tae-Wan Kim

Contents

1 Acknowledgments	3
2 Abstract	4
3 Introduction	5
3.1 The Team	7
3.2 Tools Used	8
3.2.1 Linux	8
3.2.2 GitHub	8
3.2.3 Latex	8
3.2.4 Scikit-learn	8
3.2.5 Optuna	9
4 Work Realized	10
4.1 Objective	10
4.2 Markov Models	11
5 Definitions	11
5.1 Markov Chain	11
5.2 Hidden Markov Model (HMM)	12
6 Classic HMM	13
7 Market Observations	14
8 Parameter Determination	16
9 Forward Algorithm	16
10 Gaussian HMM	17
10.1 Expectation	17
10.2 Maximization	17
11 Variational Bayesian Inference (Mean Field Approximation)	19
11.1 The Impossible Integral	19
11.2 V-E Step (Variational Expectation)	20
11.3 V-M Step (Variational Maximization)	20
11.4 Priors	20
12 State Detection	22
13 Implementation with Neural Networks	24
14 Using Optuna to Optimize Training	26
15 Training	28
16 Results	29

17 Strategy Performance Using Neural Network Implementation	30
18 Strategy Performance Using Only Hidden Markov Model	31
19 Real-Time Work	32
19.1 TradeStation API	32
19.2 Real-Time Implementation	33
20 Future Work	34
21 Conclusion	34
22 References	37

1 Acknowledgments

I would like to thank INSA Lyon for this internship opportunity, for publishing the partner offer, and for allowing me to go abroad to carry out this internship at the Seoul National University laboratory.

I also thank Professor Tae-Wan Kim, who accompanied me as my internship supervisor throughout the duration of my experience. I also thank the entire laboratory team with whom I was able to discover the professional world as well as Korean culture.

2 Abstract

Hidden Markov Models constitute a versatile class of statistical models for various data, particularly time series. It is a powerful machine learning method for detecting regimes within a class of data. Combining these Markov models with neural networks also allows for increased detection accuracy.

In financial applications, these regimes can often be linked to bear or bull markets, thus creating trading opportunities.

During this internship at the Deep Learning Laboratory of Seoul National University, we had the chance to study this fascinating subject and to develop different methods related to Hidden Markov Models in order to create a real-time trading agent capable of positioning itself on a market according to the current regime.

3 Introduction

The laboratory is located at Seoul National University (SNU), in the Gwanak district, south of Seoul, Korea. The university is organized into different research and teaching departments. These departments are grouped within different faculties, such as the College of Natural Sciences, focused on physics, chemistry, biology, and mathematics; the College of Social Sciences, dedicated to fields like economics, sociology, and political science; and the College of Humanities, which studies languages, literature, and philosophy.



Figure 1: Map of SNU

Furthermore, [SNU](#) possesses specialized schools such as the College of Medicine, the College of Business Administration, and the College of Education, which offer different research training programs. SNU has the privilege of housing one of the largest university libraries in Asia, which holds over 5.2 million books, 260,000 academic journals, and 230,000 digital resources.

The Kyujanggak Institute for Korean Studies, originally founded as the royal library of the Joseon Dynasty, is managed by Seoul National University (SNU) and constitutes a major center for research in Korean history.

As of 2022, SNU has approximately 28,264 students, including 15,870 undergraduates, 12,394 master's students, and 4,019 doctoral students. The university employs approximately 2,278 faculty researchers, under the leadership of President Ryu Hong-lim. SNU is spread across four main campuses:

- **Gwanak Campus** (main campus) - located in Gwanak-gu, Seoul
- **Yeongeon Campus** - Colleges of Medicine, Dentistry, and Nursing
- **Pyeongchang Campus** - specialized in agriculture and life sciences
- **Siheung Campus** - a global research and innovation hub in development



Figure 2: Gwanak Campus

Together, these campuses cover nearly 7 km², making Seoul National University (SNU) one of the largest universities in South Korea. The university has 16 colleges, a general graduate school offering 99 programs, and 12 professional graduate schools.

The main colleges include the College of Humanities, Social Sciences, Natural Sciences, Business Administration, Education, Medicine, Pharmacy, Agriculture and Life Sciences, Fine Arts, Music, and the College of Engineering.

The latter plays a central role in promoting technological innovation and includes departments such as Mechanical Engineering, Electrical and Computer Engineering, Civil and Environmental Engineering, Chemical and Biological Engineering, and Materials Science and Engineering.

Research conducted within the College of Engineering integrates advanced scientific principles with practical applications aimed at solving real-world challenges. Among all these departments, the internship took place within the College of Engineering, specialized in scientific and technological innovation. This faculty groups several departments, notably:

Mechanical Engineering, Electrical and Computer Engineering, Civil and Environmental Engineering, Chemical and Biological Engineering, Materials Science and Engineering, and many others.

These departments are heavily involved in engineering and research projects, contributing significantly to industrial and societal development.



Figure 3: Laboratory Building

3.1 The Team

The laboratory team was mainly composed of Ph.D. students, interns, and engineers. The director of this laboratory is Professor Tae-Wan Kim, who acted as our supervisor. He was responsible for providing advice and feedback on our weekly work and the direction of our research. In the workspace, Jae Hyun Kim, a Ph.D. student, was the lab manager. He supervised laboratory maintenance and the daily management of tasks and needs of workers for their projects. Other members of the laboratory worked in various specialized research fields, contributing to a diverse and collaborative environment.

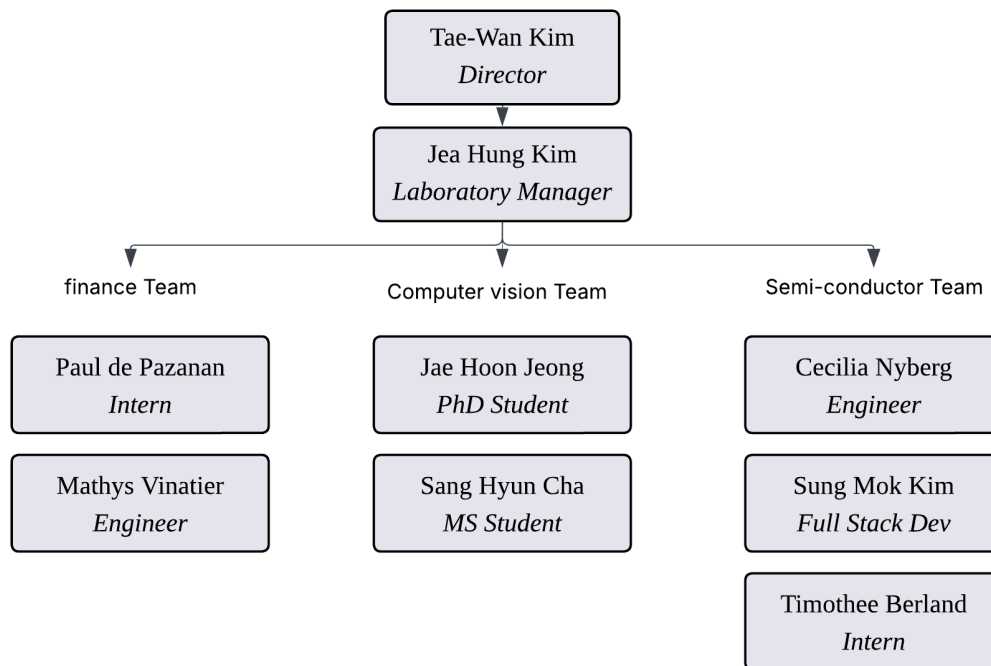


Figure 4: Laboratory Organization

The computer vision team was in charge of developing a multi-agent Large Language Model (LLM) and a Simultaneous Localization and Mapping (SLAM) system for object recognition in maritime environments to ensure safe navigation. This team regularly performed field tests to evaluate their hardware and applications. They possessed strong expertise in Deep Learning and AI and often exchanged ideas with the financial team through discussions on how AI could be introduced and its limitations.

The semiconductor team focused on developing hardware for AI inference, specifically for LLM-based agents. Their mission was to design FPGA-based embedded systems capable of executing heavy computational models efficiently. Engineers in this team worked on compiling and optimizing deep learning models like DeepSeek chatbot applications so that FPGA boards could execute them independently with limited computing resources.

As for me, I was part of the financial team where my main role was to filter and analyze the market to allow obtaining different market regimes and then to be able to implement

a prediction agent by dynamically adjusting filters and indicators to automate trading decisions using scripts. I also focused on signal processing and smoothing techniques to identify the most suitable solutions. An engineer from the team was in charge of developing a Proximal Policy Optimization (PPO) algorithm so that I could implement my HMM as an actor.

3.2 Tools Used

During this internship, we used different tools to develop this algorithm and to simulate it. We used the Python programming language to write all our codes, as well as other tools.

3.2.1 Linux

During the internship, we used the Linux operating system to work on both workstations and remote servers. Throughout the project, we used Linux Ubuntu as the main desktop environment and Ubuntu servers to manage the various services and training sessions requiring execution on the laboratory infrastructure. I knew Linux very briefly but had never used it so intensively on a server before.

During the development of the internship project, Linux proved to be extremely suitable, as it allowed us to manage computer resources very efficiently, particularly regarding GPU usage, process allocation, and system monitoring. Moreover, most artificial intelligence development tools are optimized for Linux systems, which greatly facilitated low-level debugging and resolving environment issues.

3.2.2 GitHub

We used Git throughout this internship, notably to keep a reliable copy of our work and to facilitate access to our scripts for collaborative purposes. We also used Git to automatically publish our deep learning models to ensure their continuous backup, as their training was extremely long and we could not afford to lose any progress.

3.2.3 Latex

We used Latex for writing purposes for our reports; it was necessary to continuously write and summarize the work we were doing so that the manager could guide us adequately on the work to be done next.

3.2.4 Scikit-learn

The key features of this library are: classification, regression, clustering, dimensionality reduction, and data preprocessing. This library is popular because whether one uses linear regression or a complex neural network, the commands are the same: `.fit()` and `.predict()`.

Scikit-learn is the industry-standard library for Machine Learning in Python. It is built on other high-performance scientific libraries like NumPy, SciPy, and Matplotlib. It provides simple and efficient tools for data analysis and modeling.



Figure 5: sklearn Logo

3.2.5 Optuna



Figure 6: Optuna Logo

Optuna is a Python library used for task optimization. We used it to fine-tune the hyperparameters of our model. With Optuna, we can create an automated loop that runs our model training while saving the different results and evaluation metrics of each model in a database. Optuna also has its own internal database, where it uses advanced search algorithms, such as grid search and more sophisticated sampling methods, to explore the space efficiently and converge towards the defined optimization.

4 Work Realized

4.1 Objective

We have seen the tools used and an introduction to the enterprise. Let us now focus on the work actually realized.

The goal of the internship was to provide reliable signals for algorithmic trading on a very specific market, the VIX Index, and to be able to trade this index using [Futures](#). The figure below shows us the price of the VIX Index since 2000. We can see that its price oscillates between 0 and 100, as this index measures the expected 30-day volatility of the S&P 500, or in other words, investor fear. The higher the price, the higher the volatility, representing high fear among investors.

We will therefore develop a two-part tool allowing us to position ourselves in this market. First, develop an algorithm allowing us to filter the market and analyze which regime we are currently in (bull, bear, or neutral regime). Then, we will need to integrate the created model with neural networks to improve detection accuracy and produce reliable buy or sell signals.

Furthermore, we will also need to develop different simulation tools allowing us to test our methods in real-time, as the final goal is to be able to deploy our algorithm on a real market.

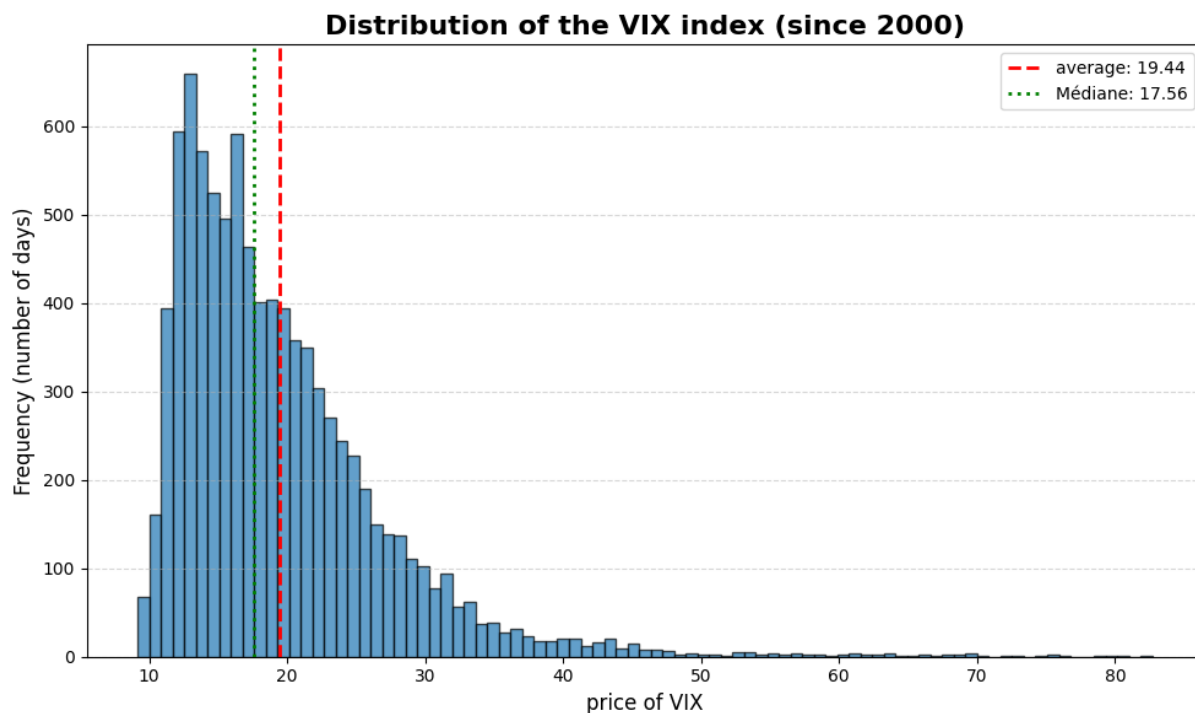


Figure 7: Distribution of VIX prices since 2000

4.2 Markov Models

To achieve this, we used Hidden Markov Models (HMM).

Indeed, this choice appeared relevant because these models constitute a versatile class of statistical models for different data, and particularly time series. Market data are time series, and sometimes very noisy, which makes it difficult to detect apparent regimes with the naked eye. This is why HMMs appeared as a natural choice allowing to determine different states in a stationary financial market (The VIX index) where states can be complicated to spot manually.

5 Definitions

5.1 Markov Chain

In order to understand Hidden Markov Models (HMM), it is first necessary to study Markov chains. A Markov chain is a stochastic model that represents a sequence of events or states. Let S be a finite or countable set and (X_n) a stochastic process with values in S . Then X is a Markov chain if, for all $n \in \mathbb{N}$ and for all $x_0, x_1, \dots, x_{n+1} \in S$ such that $P(X_0 = i_0, \dots, X_n = i_n) > 0$, we have

$$P(X_{n+1} = i_{n+1} \mid X_0 = i_0, \dots, X_n = i_n) = P(X_{n+1} = i_{n+1} \mid X_n = i_n) \quad (1)$$

This means that the probability of transitioning to a certain state at time $n + 1$ depends only on the current state at time n and not on previous states.

A Markov chain is defined by the set of states S and by a transition probability matrix $A = [a_{ij}]$, where a_{ij} represents the probability of moving from state S_i to state S_j in a single step.

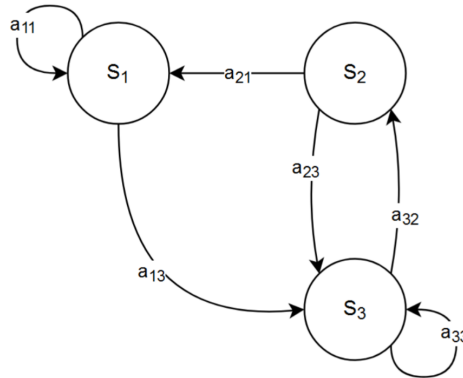


Figure 8: Markov Chain with 3 states

Let $x_n \in \mathbb{R}^N$ be the vector containing the probabilities of being in each of the states at time n . The evolution of the system is then given by

$$x_{n+1}^T = x_n^T A \quad (2)$$

5.2 Hidden Markov Model (HMM)

Unlike a classic Markov chain where states are visible and observable, in a Hidden Markov Model (HMM), the states are hidden and are deduced from observations. The hidden state process of an HMM is a Markov chain.

Let

$$O = \{O_k \mid O_k \in \mathcal{O}, k = 1, \dots, T\}$$

be an observed sequence, where $\mathcal{O}$ denotes the set of possible observations. The hidden states evolve according to equation (2).

The structure of an HMM is defined by its different characteristics:

- λ is the set of model parameters;
- N is the number of hidden states in the model. The set of hidden states is $S = (S_1, \dots, S_N)$ and the hidden state at time t is denoted Y_t ;
- M is the number of distinct observable symbols per state. These symbols are denoted o_k with $k = 1, 2, \dots, M$ and the observation at time t is denoted O_t ;
- $A = \{a_{ij}\}$ is the transition matrix of hidden states, defined by:

$$a_{ij} = P(Y_{t+1} = S_j \mid Y_t = S_i), \quad i = 1, 2, \dots, N, j = 1, 2, \dots, N;$$

- $B = \{b_i(o_k)\}$ is the matrix of observation probabilities of symbol o_k when the system is in state S_i , defined by:

$$b_i(o_k) = P(O_t = o_k \mid Y_t = S_i), \quad i = 1, 2, \dots, N, k = 1, 2, \dots, M;$$

- $\mu = \{\mu_i\}$ is the probability distribution of the initial state of the model, defined by:

$$\mu_i = P(Y_1 = S_i), \quad i = 1, 2, \dots, N.$$

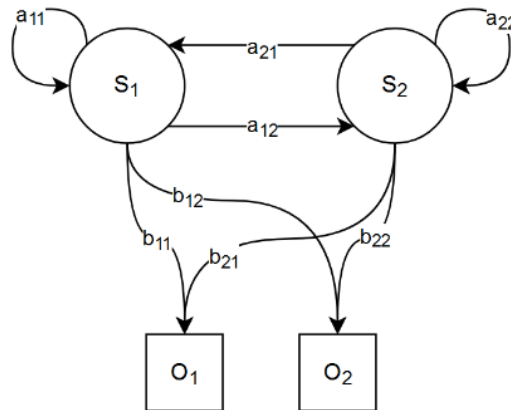


Figure 9: Hidden Markov model chain with 2 states and 2 possible outputs

6 Classic HMM

The HMM is therefore a relevant method for detecting different regimes in a financial market where regimes are not always observable.

We used the VIX index and defined a three-state HMM, assuming that the VIX Index is stationary and will always oscillate between 0 and 100. We thus define a normal state, where the index is around its average price without buying or selling opportunities, a bottom state, where the index is oversold and thus triggers a buying opportunity, and finally a top state, where the index is overvalued and thus triggers a selling opportunity.

As we saw earlier, these three states are hidden, because we cannot determine them directly: we need market observations. This hidden state process is defined by a simple Markov chain, as shown in figure 10 with its red part.

The model will therefore take market observations, which we will define later, and which will allow us to determine our hidden states, as represented in figure 10 with the green part. The model is composed of three important parameters whose determination we will study later: a transition matrix, which represents the probability of moving from one state to another within the Markov chain, as well as an emission matrix, which represents the probabilities of being in a certain state given the current observation. Finally, there are the initial probabilities, which are to be determined.

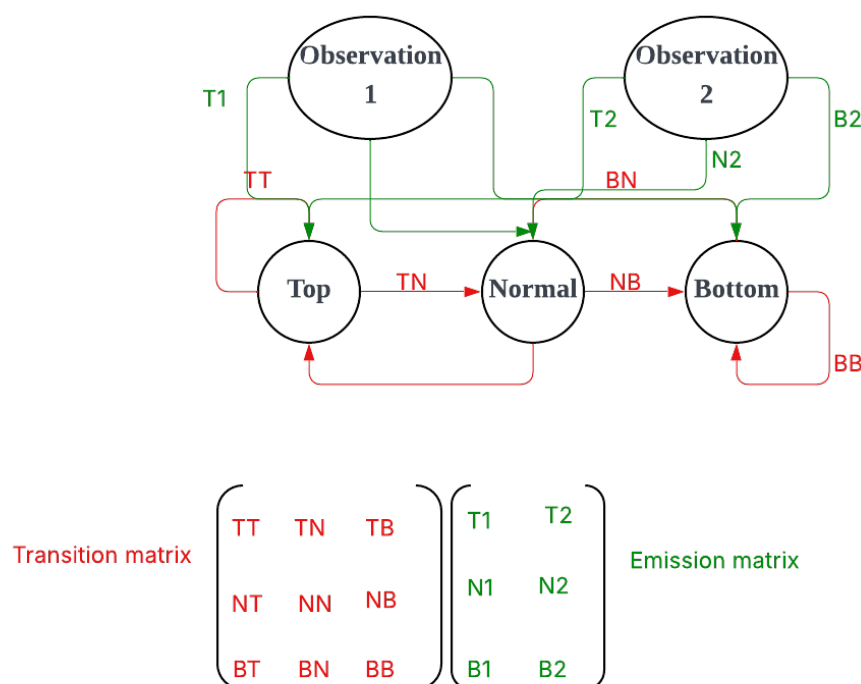


Figure 10: Three-state HMM using 2 observations

The goal of the HMM is therefore, given a sequence of observations, to find the most probable sequence of states. As represented in figure 11, if we have the sequence of observations 1 then 1 then 2, we must find the most probable sequence of states using the emission and transition matrices. The process will be explained subsequently.

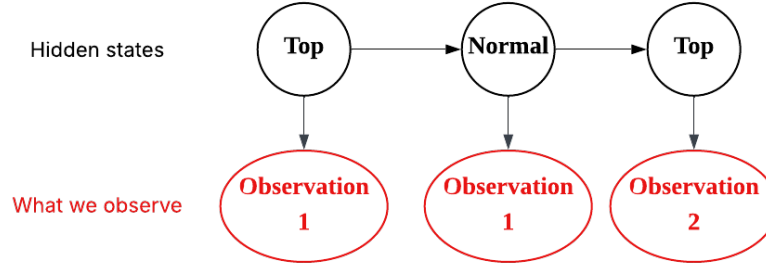


Figure 11: Hidden Markov model chain with 2 states and 2 possible outputs

7 Market Observations

To be able to spot our different regimes, we therefore need market observations. On the previous graph, two observations were represented; in reality, we will need many more to gain precision. However, we must also avoid using too many, as our model might not spot patterns if too much data is provided, and thus perform overfitting.

We obtain, at each time period t , the following matrix containing market price information.

$$state = \begin{pmatrix} open_1 & close_1 & low_1 & high_1 & volume_1 \\ open_2 & close_2 & low_2 & high_2 & volume_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ open_t & close_t & low_t & high_t & volume_t \end{pmatrix}$$

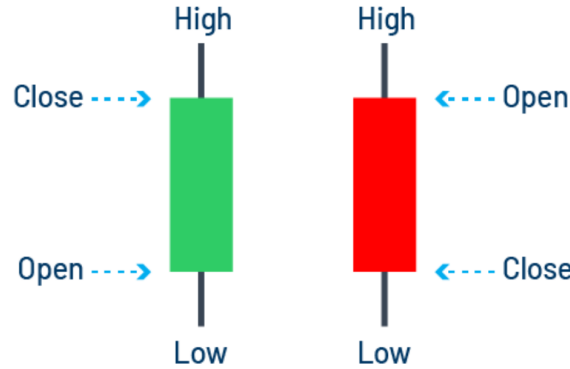


Figure 12: Values defining a candle

From this matrix, we can calculate useful data representing the market state; we will call these data observations. These observations can be more or less correlated with each other (figure 12) and are already at the origin of different trading strategies. Throughout our study, we will select from these data and will not always use all observations. The four most important features are the following:

1. Log Returns:

$$\logret_t = \ln \left(\frac{P_t}{P_{t-1}} \right) \quad (1)$$

2. **Z-Score** (Normalization over 20 periods):

$$\text{zscore}_t = \frac{P_t - \mu_{20}}{\sigma_{20}} \quad (2)$$

Where μ_{20} is the moving average and σ_{20} is the standard deviation over 20 days.

3. **Relative Range** (Relative volatility):

$$\text{range}_t = \frac{\text{High}_t - \text{Low}_t}{P_t} \quad (3)$$

4. **Slope10** (Momentum slope):

$$\text{slope10}_t = \frac{P_t - P_{t-10}}{10} \quad (4)$$

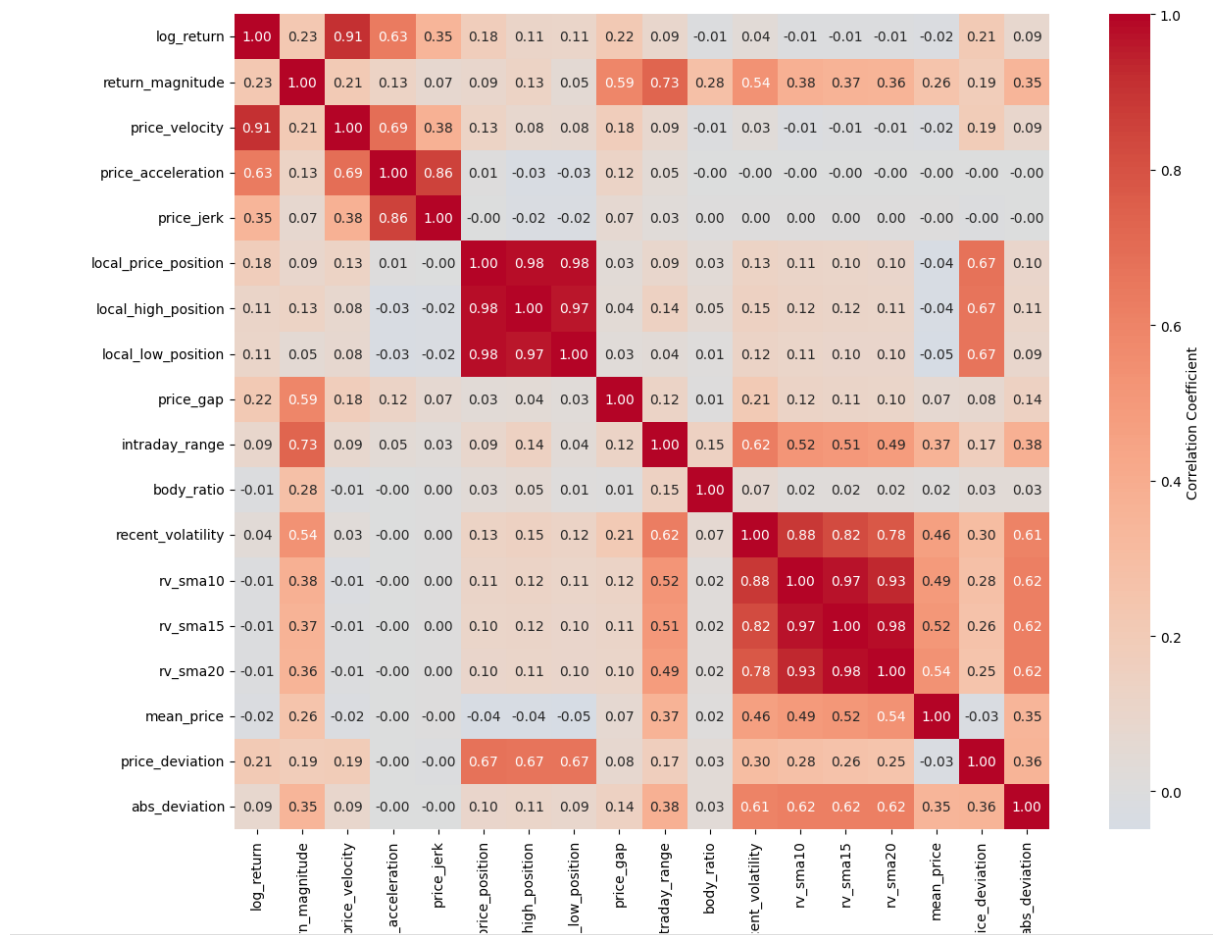


Figure 13: Correlation matrix of different observations

8 Parameter Determination

We have seen the structure of an HMM model and defined the observations that will allow us to identify the different states. We must now determine a method to find these variables. There are several paths to achieve this:

- Use heuristic emissions and define the HMM manually ourselves, updating the parameters using Bayes update.
- Use the classic method with the Baum-Welch algorithm and a Gaussian model to train the model on a dataset to directly obtain the parameters.
- Use a more advanced method with Variational Bayesian Inference.

We therefore seek to determine the 3 parameters governing the HMM:

- The initial probabilities
- The transition matrix
- The emission matrix

9 Forward Algorithm

Once our parameters are determined, we simply need to apply a forward algorithm to be able to determine in real-time the state in which we currently are. The formula below summarizes this: at each instant of time t , we use the emission matrix, the transition matrix, as well as the previous probabilities to determine the current state.

$$P(Z_t = k | X_{1:t}) \propto P(X_t | Z_t = k) \cdot \sum_{j=1}^K P(Z_t = k | Z_{t-1} = j) \cdot P(Z_{t-1} = j | X_{1:t-1})$$

Figure 14: Forward algorithm for the test period

10 Gaussian HMM

A first method to obtain these parameters is to apply the simple model of a Gaussian HMM, the most widespread one. This one will return a unique value for each probability of each parameter. We must therefore find the parameters that maximize the probability of observing our sequence of data O . To do this, we will use the Baum-Welch algorithm (or Expectation Maximization Algorithm), which works in two steps.

10.1 Expectation

In this step, we randomly initialize the parameters we are looking for in order to estimate the probability of being in a certain state at a given instant.

We will perform a forward-backward algorithm on the training data to calculate 2 indispensable intermediate variables for each instant t :

- $\gamma_t(i)$: The probability of being in state i at time t , given the entire observation sequence.
- $\xi_t(i, j)$: The probability of moving from state i at time t to state j at time $t + 1$.

10.2 Maximization

Now that we have determined these estimates, we update the model parameters so that they best correspond to these estimates.

The update formulas are quite intuitive, as they are weighted averages:

Transition update (a_{ij}):

$$\hat{a}_{ij} = \frac{\text{Expected number of transitions from } i \text{ to } j}{\text{Expected number of transitions leaving } i} = \frac{\sum_t \xi_t(i, j)}{\sum_t \gamma_t(i)}$$

Emission update (discrete case) ($b_j(k)$):

$$\hat{b}_j(k) = \frac{\text{Expected number of times being in } j \text{ and observing } k}{\text{Expected number of times being in } j}$$

The explanation above assumes that values are discrete. But in reality, in finance, values are continuous. In that case, the emission matrix B cannot be just a probability table. We replace this table with a probability density function. The most popular is a Gaussian (normal distribution).

Thus, for the maximization step, we will have to learn the parameters of the Gaussian function instead of learning discrete probabilities:

- The Mean (μ_j): The "center" of observations for this state.
- The Covariance (Σ_j): The "dispersion" or shape of the point cloud for this state.

The Baum-Welch algorithm adapts very well to this and we therefore learn the following new parameters:

New Mean ($\hat{\mu}_j$), weighted average of all observations o_t , where the weight is the probability that this observation comes from state j .

$$\hat{\mu}_j = \frac{\sum_t \gamma_t(j) \cdot o_t}{\sum_t \gamma_t(j)}$$

New Covariance ($\hat{\Sigma}_j$), weighted variance of observations relative to the new mean.

$$\hat{\Sigma}_j = \frac{\sum_t \gamma_t(j) \cdot (o_t - \hat{\mu}_j)(o_t - \hat{\mu}_j)^T}{\sum_t \gamma_t(j)}$$

This training will allow us to find exact values for our parameters. The algorithm process is represented below, figure 15:

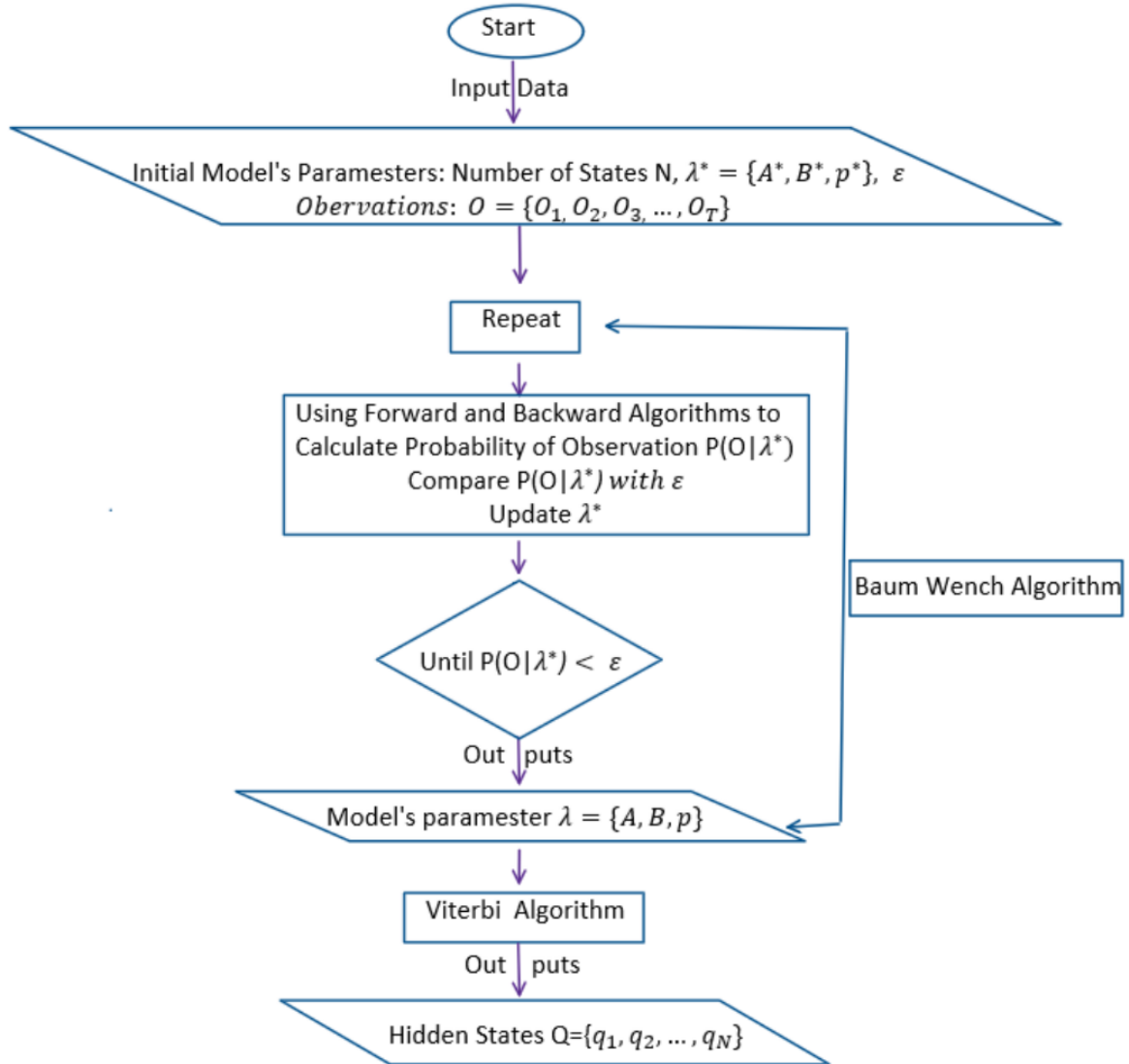


Figure 15: HMM training and detection process

This method is simple to implement, will always converge to a local optimum, and can easily adapt to discrete or continuous emissions. However, it also presents drawbacks that we will analyze in the next part.

11 Variational Bayesian Inference (Mean Field Approximation)

The biggest drawback of the classic Baum-Welch method is that the estimates given are point estimates: the algorithm gives us a single precise value for each parameter (e.g., $\mu = 0.5$). It gives us no idea of the uncertainty regarding this value. In finance, this is a major problem, because if we have little training data for a certain state, our model will return a single value and we will have no information on whether this value is solid or not.

Furthermore, if we have little data, the model will still learn parameters, but they will not be precise at all, and this will cause overfitting, and results in the test period will be much worse.

To solve this problem, we need to improve detection accuracy. Indeed, with the classic HMM, to move from state 1 to 2, there is a unique probability, for example, 0.3. But if the model has not had time to learn well, this value will not be truly representative. Thus, to solve this, we will use a Bayesian HMM, specifically Variational Bayesian Inference. This second method consists, instead of estimating a single point value, $\theta^* = \arg \max_{\theta} p(X | \theta)$, of seeking to approximate the complete posterior distribution of parameters:

$$p(\theta | X) \approx q(\theta),$$

11.1 The Impossible Integral

To find our distribution, we must rigorously apply Bayes' rule, which involves calculating a denominator requiring an integral over all possible parameter configurations. The problem is that, in a complex HMM, this integral is impossible to calculate. We will therefore use a variational approach.

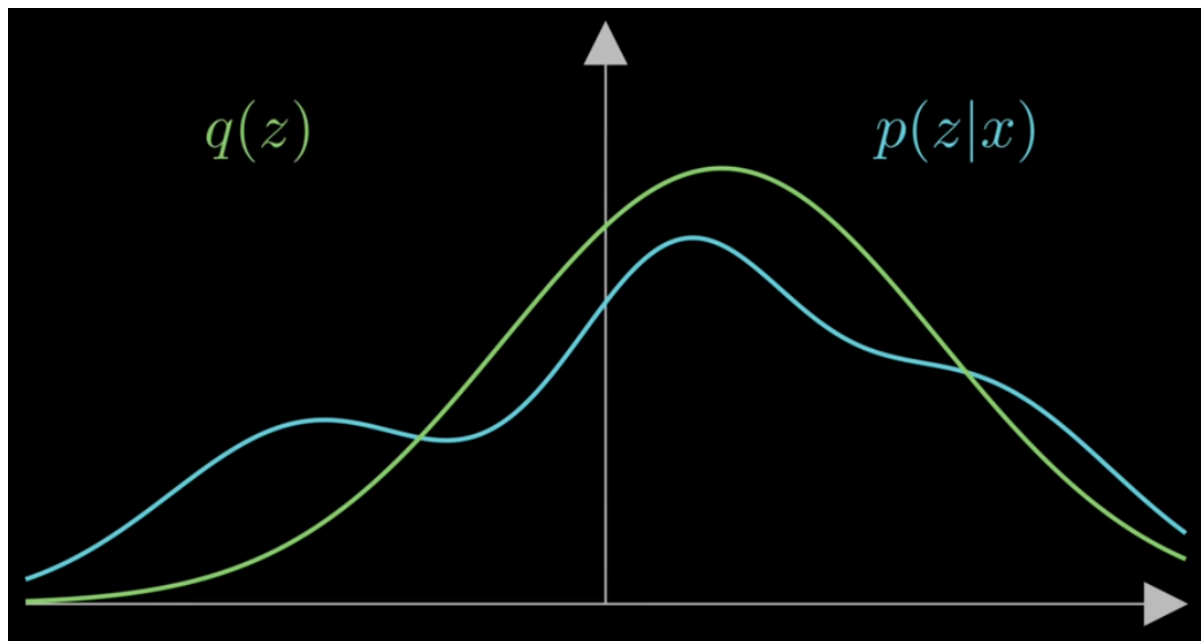


Figure 16: Representation of the variational approach

As we see in Figure 16, instead of calculating the precise distribution $p(z|x)$, we will approximate this distribution and find $q(z)$.

To achieve this, we will use an algorithm similar to Baum-Welch: the Variational Bayesian EM.

11.2 V-E Step (Variational Expectation)

As in the classic approach, we use the Forward-Backward algorithm, but instead of using the raw values of parameters A and B , we will take their expectation (the mean of their current distributions).

We thus obtain the sufficient statistics (γ and ξ) which tell us which states are probable.

11.3 V-M Step (Variational Maximization)

We update the "hyper-parameters" of the q distributions.

For the transition matrix, it is very popular to use a Dirichlet distribution. We update its counters by adding the observed transitions in step E.

For Gaussian emissions, we use a Normal-Wishart distribution. We update the mean and the precision matrix based on the weighted data.

11.4 Priors

Unlike the classic HMM, we therefore place conjugate prior distributions on all HMM parameters, which was not done before:

- Initial distribution:

$$\pi \sim \text{Dir}(\alpha_0).$$

- Rows of the transition matrix:

$$A_i \sim \text{Dir}(\alpha_{A0}) \quad \text{for all } i.$$

- Gaussian emission laws (for each state k and each dimension d):

$$\tau_{kd} \sim \text{Gamma}(a_0, b_0) \quad (\text{precision}),$$

$$\mu_{kd} \mid \tau_{kd} \sim \mathcal{N}(\mu_0, (\lambda_0 \tau_{kd})^{-1}).$$

After using the algorithm taking the market observations defined previously, we finally obtain the following parameters for our hidden Markov model:

$$\text{Initial probabilities } \boldsymbol{\pi} = (0.49990132 \quad 0.25006805 \quad 0.25003063)$$

$$\text{Transition matrix } \mathbf{A} = \begin{pmatrix} 0.91315907 & 0.02261674 & 0.06422420 \\ 0.07877199 & 0.83853480 & 0.08269320 \\ 0.05266225 & 0.01839238 & 0.92894537 \end{pmatrix}$$

$$\text{Emission matrix } \boldsymbol{\mu}_{\text{mean}} (K \times D) = \begin{pmatrix} 8.660 \times 10^{-4} & 1.046 & 4.723 \times 10^{-3} & 1.629 \times 10^{-2} \\ 6.023 \times 10^{-4} & 0.1622 & 1.492 \times 10^{-2} & 1.158 \times 10^{-2} \\ -8.411 \times 10^{-4} & -1.037 & 4.210 \times 10^{-3} & -1.587 \times 10^{-2} \end{pmatrix}$$

$$\sqrt{\boldsymbol{\Sigma}_{\text{mean}}} (K \times D) = \begin{pmatrix} 0.00623040 & 0.73597400 & 0.00578912 & 0.01795274 \\ 0.02114252 & 1.77368075 & 0.01631164 & 0.15907400 \\ 0.00552426 & 0.70326405 & 0.00515582 & 0.01756078 \end{pmatrix}$$

In summary, we therefore have better chances of performance using Variational Bayesian Inference. We will see later that our results were better with VC EM. The table below allows us to summarize the difference between these two methods:

Characteristic	Baum–Welch (Classic EM)	Variational Bayes (VB)
Object sought	Fixed values (point estimates)	Probability distributions
Mathematical objective	Likelihood maximization	ELBO maximization
Overfitting management	Weak (high risk)	Excellent (intrinsic regularization)
Complexity	Simple, fast	More complex, mathematically heavy

Table 1: Comparison between Baum–Welch and Variational Bayes

12 State Detection

We now add to the regime detection a method to be able to produce buy and sell signals in order to look at the robustness of our models. We introduce another parameter, which is the outlier line. Indeed, at each instant t , our model returns a certain probability of being in each state, but to have a buy signal, it does not necessarily have to be the one with the highest probability: sometimes, one should just do nothing. Thus, for this, we use this outlier line which functions as a threshold and which represents an abnormally high or low price. Thus, if our probabilities cross this line, this will trigger a buy or sell signal, as represented in figure 17.

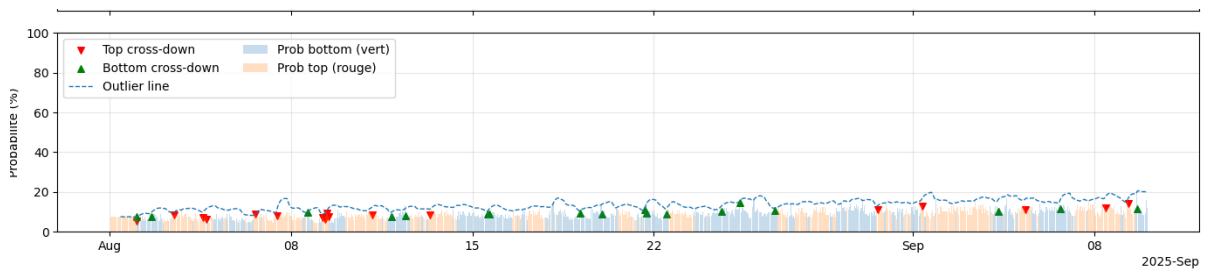


Figure 17: Trigger process of a signal

Our classic HMM model allowed us to spot states well in a large time interval (daily): this means that each candle represents a day. But when we went down to a smaller time interval of 5 min (each candle represents 5 minutes), then the model could no longer learn correctly and detection became difficult. This is why we had the disappointing results of figure 18 below.

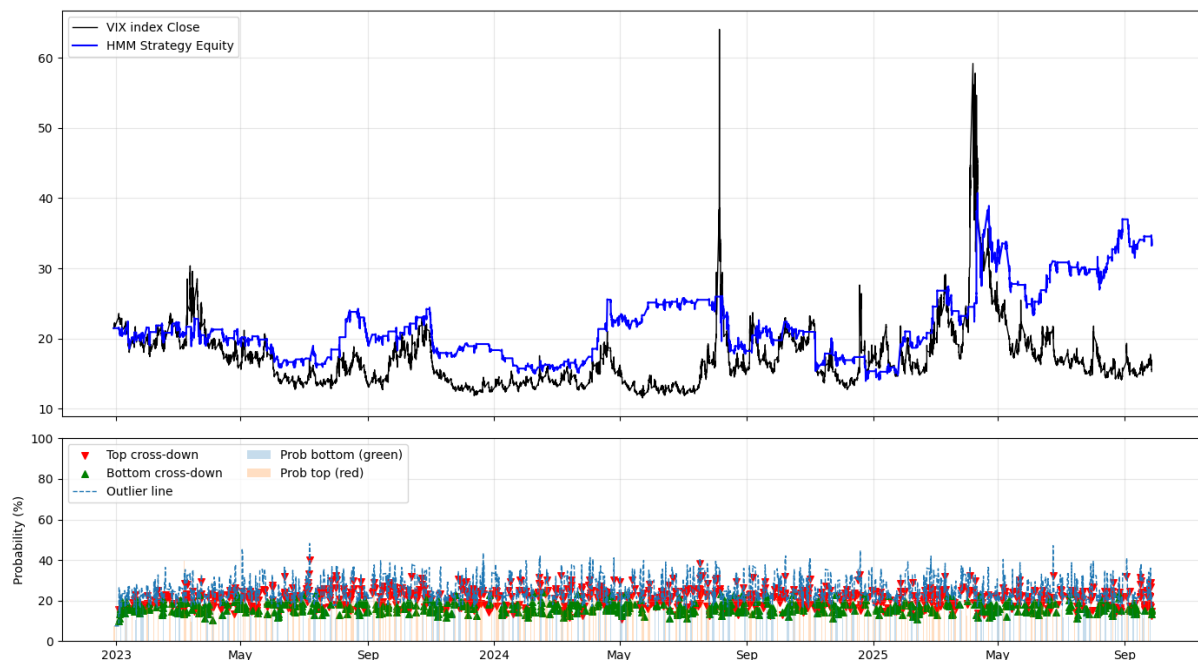


Figure 18: Classic HMM Results

However, we desire to work in a small time interval because large fees apply in reality if we want to hold a position on [Futures](#) for a long time. Indeed, every month the contract

expires and a new one is issued. If we want to keep the position, we must automatically sell and buy back but at a higher price, which produces a loss.

Thus, we see that the results obtained with the Bayesian HMM on the figure below are much better even if large drawdowns are present sometimes; this will be resolved later.

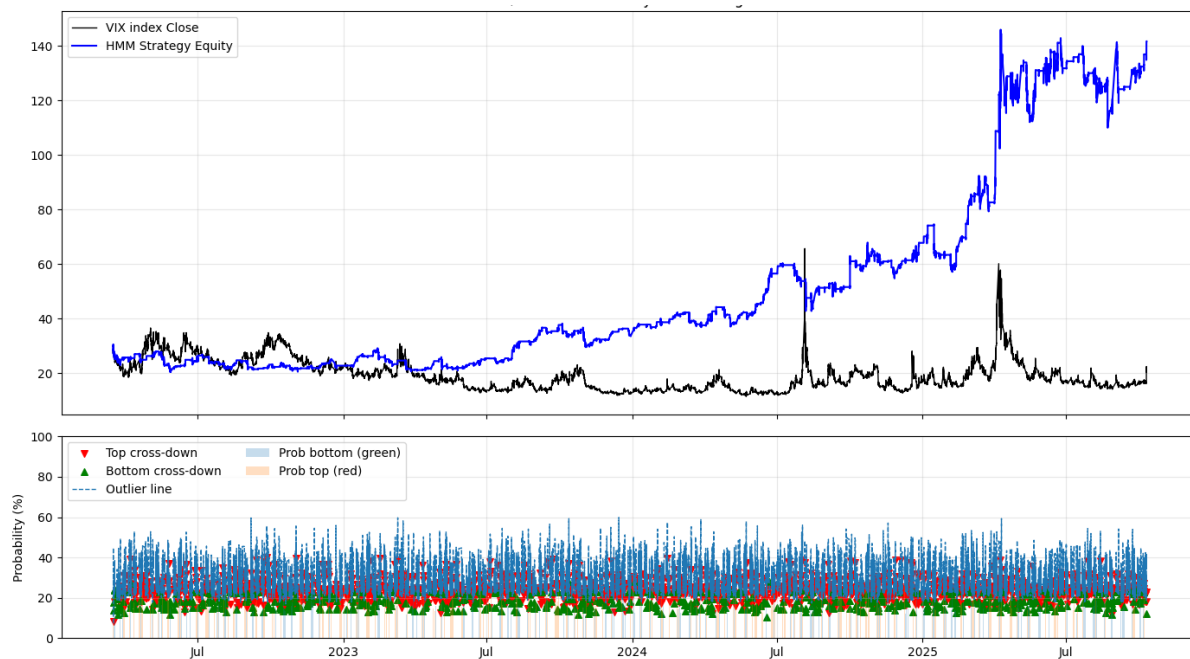


Figure 19: Bayesian HMM Results

13 Implementation with Neural Networks

Our Bayesian [HMM](#) therefore now returns better regime detections than the classic HMM, but we now want an actor that allows us to return buy and sell signals with precision. Because although our model produces good regimes, false positives remain. We therefore want to eliminate these errors; for this, we will use an architecture composed of neural networks that will also learn patterns in the market and concatenate with the probabilities returned by the HMM to be able to return a precise final probability of rise (or fall).

In our architecture, we will use [LSTM](#) and [MLP](#). Here is figure 20, the detailed description of our training organization.

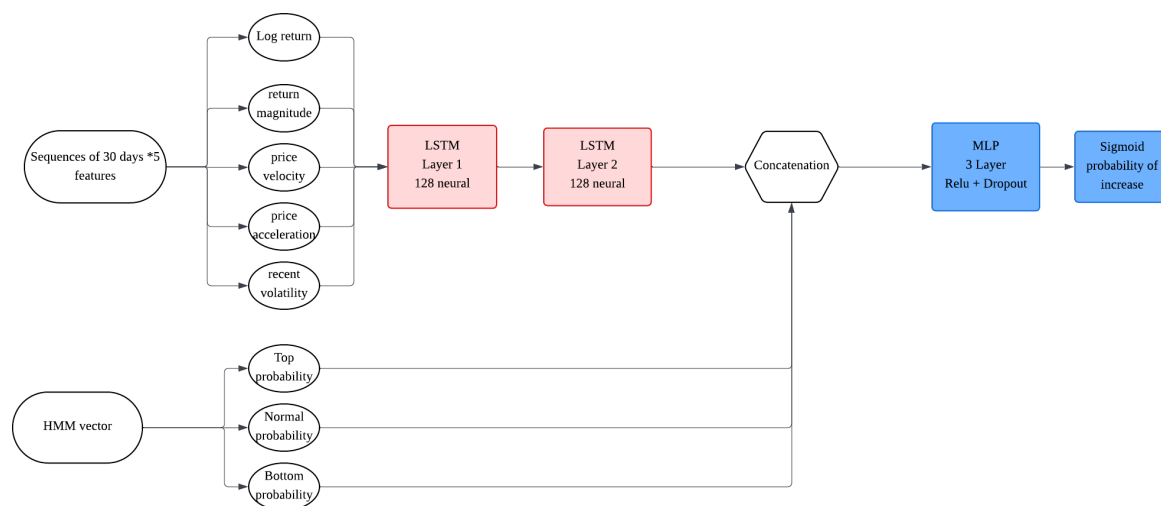


Figure 20: Training Architecture

Our HMM detailed previously therefore returns the 3 refined regime probabilities.

The [LSTM](#), unlike a classic neural network, allows for understanding temporal dependencies. Thanks to its "memory" modeled by its different input, output, and forget gates, it can adapt and detect price patterns.

The [MLP](#) will serve as the final brain; it will take as input the hidden states returned by the HMM and the detection of the LSTM, and it will combine these 2 visions to return a robust last prediction.

To train this model, we initially used simple error functions such as [MSE](#) and [BCE](#) which only ask the model to predict the price. Results using these functions were limited because these error functions treat every error in the same way; if the prediction is wrong by 0.1% or 5%, our model will assume that we made the same error, which is not optimal at all.

We therefore finally improved our model by using the **SharpeLoss** objective function which, unlike BCE, aims to maximize return but adjusted for risk. For this, our objective function calculates the [sharpe ratio](#). It needs 2 values:

- Confidence (y_i): The probability of rise given by the LSTM which represents the size of our position
- Returns (R_i): The real market return observed later

Our model therefore calculates the strategy return

$$R_{\text{strat},i} = R_i * y_i$$

Having this, we will now calculate the mean and volatility of these returns over our entire batch (which is the current group of data)

$$\mu = \frac{1}{n} \sum_{i=1}^n R_{\text{strat},i}$$

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (R_{\text{strat},i} - \mu)^2 + \varepsilon}$$

The Sharpe ratio is defined by

$$\text{Sharpe} = \frac{\mu}{\sigma}$$

Finally, in Deep Learning, the loss function always seeks to minimize a value, so since we want to maximize the Sharpe ratio, the loss function will be the inverse of the Sharpe ratio.

This architecture allows us to improve our signals, to gain precision, and to drastically reduce false positives. This model therefore takes fewer positions on the market but takes more precise and thoughtful positions. Indeed, it takes a position when the LSTM and the HMM are in agreement. We can see this in figure 21; our robot takes position on the confirmed buy and sell, but if only the HMM sends signals, our model ignores them:

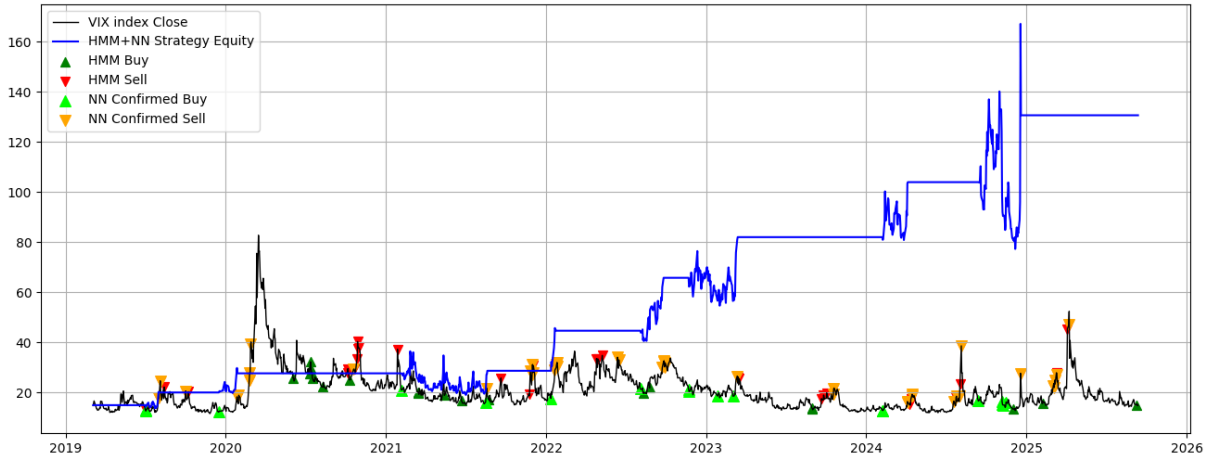


Figure 21: Visualization in test period

14 Using Optuna to Optimize Training

To carry out the previous training, we used arbitrary values for the various hyperparameters allowing for training, such as the number of epochs, the learning rate, the batch size, or the sequence duration.

Since we did not know which optimal values to provide for our model to learn well, and not wanting to test everything by hand as it would take an insane amount of time, we used Optuna. Optuna was created to automate hyperparameter optimization and can test all values for us, indicating which hyperparameters are the most important for our model to achieve better results.

We therefore launched Optuna to optimize the following hyperparameters:

- **Number of epochs:** An epoch corresponds to a complete pass of the entire training dataset through the neural network. A value that is too small can cause underfitting, while one that is too large can cause overfitting.
- **Learning rate:** This is the step size that the optimization algorithm takes to adjust the model weights in order to minimize the loss (the SharpeLoss).
- **Batch size:** The model processes data in groups; this allows adjusting the size of the different groups.
- **Sequence length** (training episode): This is the "lookback window" that the LSTM uses for each prediction.

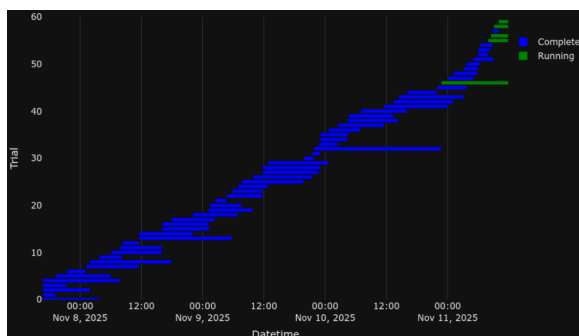


Figure 22: Different training runs

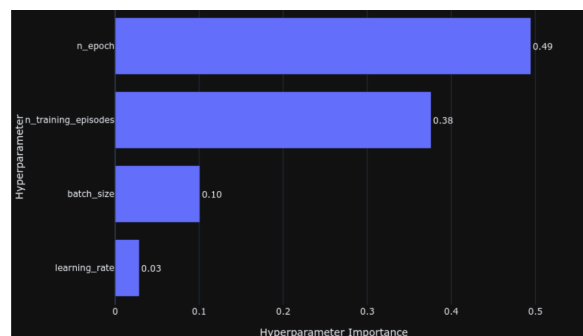


Figure 23: Hyperparameter importances

As shown in Figure 23, we see that some parameters have more importance than others; this allowed Optuna to prioritize which hyperparameters to modify during training. We finally retained the following values for our different hyperparameters, as these values offered optimal training:

Training_episodes = 90
 BATCH_SIZE = 128
 Learning_rate = 10^{-4}
 N_EPOCHS = 300

We can see below a figure representing the different equity curves during the test period over a given time interval. We see that by changing certain hyperparameters, our performance can change drastically.

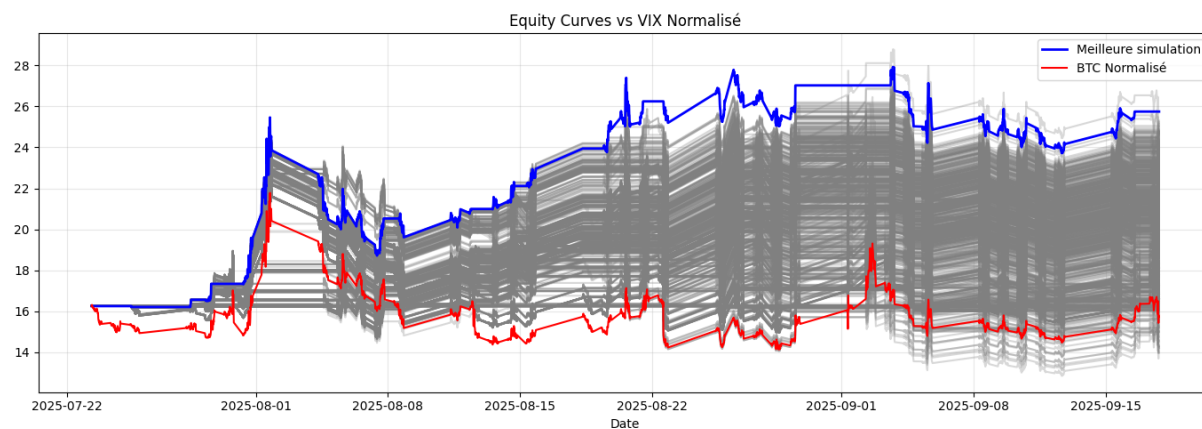


Figure 24: Visualization of optimizations with Optuna

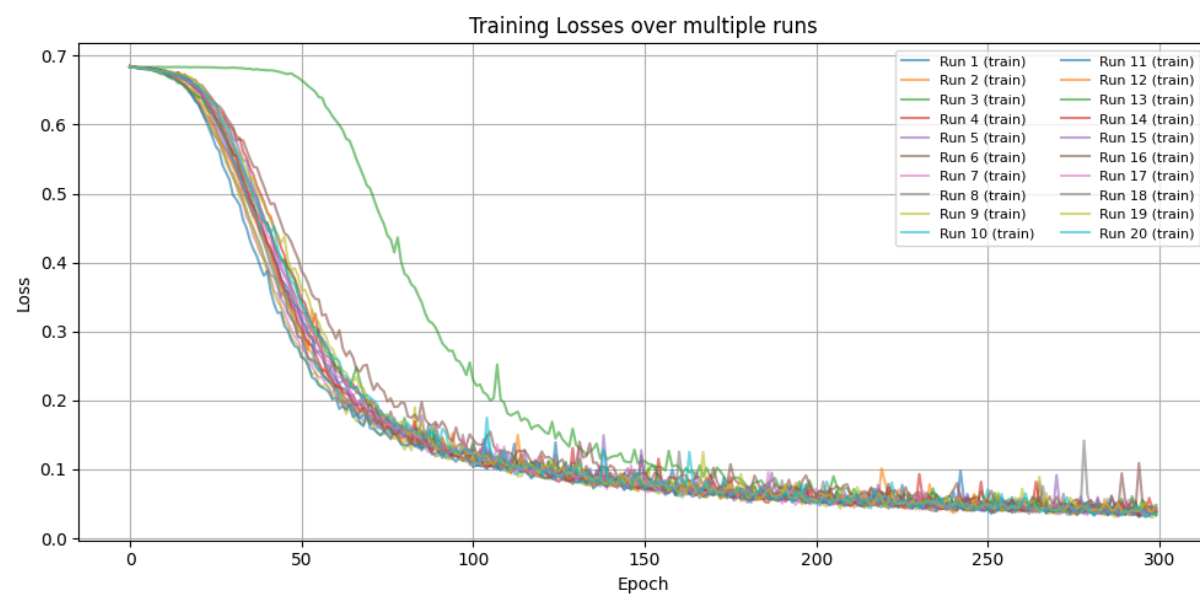


Figure 25: Loss evolution

15 Training

Having now optimized our models as best as possible, we relaunch the training sessions, also varying the timeframe on which we work. As we can see in Figure 23, we separate our data into three parts: training / validation / test.

We trained different models: Classic HMM, Variational Bayesian HMM. Furthermore, we modified the input data fed into our training: training with only VIX Index data, training with VIX Futures, the VVIX Index, or even with the S&P 500.

We also trained our models on different timeframes in order to completely study performance and to be able to determine which timeframe to use.

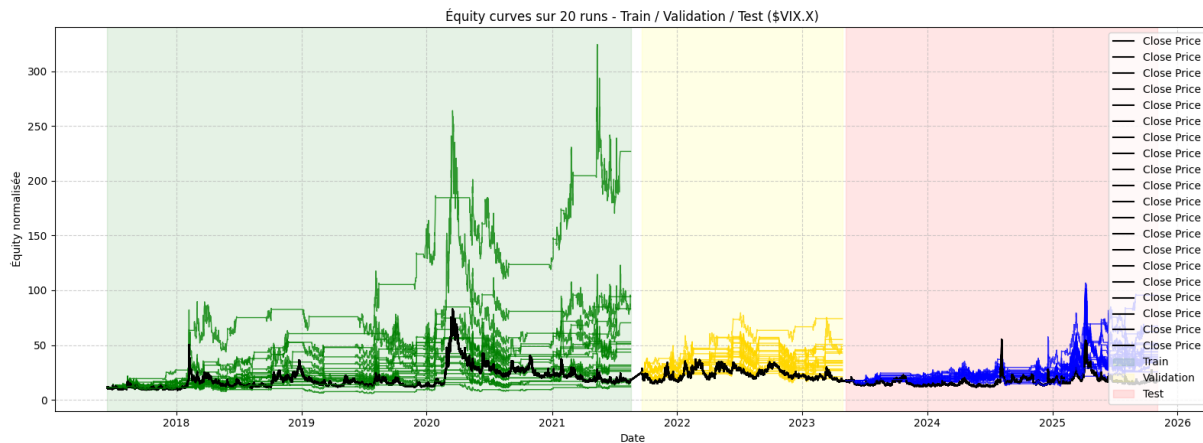


Figure 26: train/validation/test

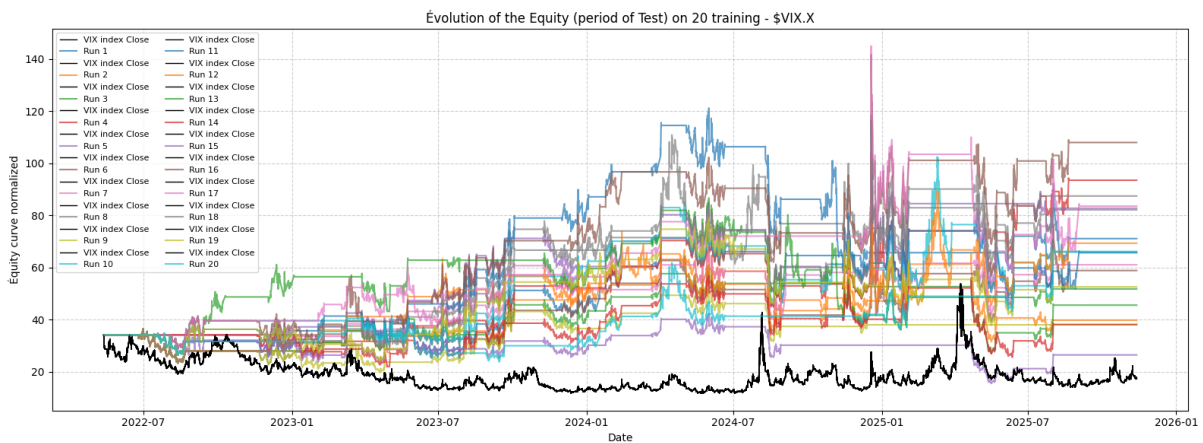


Figure 27: Test period over 20 runs

16 Results

After training our different models, we now analyze the results using several criteria, listed below. We will not describe all the trained models here, as their number is very high. We will focus only on those relevant to our study, as well as on the model selected for real-time use.

Metrics	Meaning
Total return	Total return over the entire test period in percentage or dollars
Annual return	Annual return in percentage or dollars
Annual volatility	Represents the volatility of the strategy. A high value indicates positions taken close to random chance.
Max drawdown	Represents the maximum loss observed between a peak and a trough on the equity curve.
Average drawdown	Average of all observed losses.
Timeframe	Duration of a candle.
Average profit per trade	The average profit of a trade (in percent or dollars).
Average trade duration	The average duration of a trade.

We were thus able to train our model to find the following best results during the test period. We did not simulate an equity curve in percentage but in dollars, since for each position we take, we buy a Future contract and returns are expressed in dollars for these contracts.

17 Strategy Performance Using Neural Network Implementation

	2h timeframe trading	5 min timeframe trading
Duration of the backtest	3 years	3 years
Total return	377 %	106 %
Annual return	56.85 % / \$11,370	27 % / \$5,400
Annual volatility	19 %	12 %
Sharpe ratio	2.4	1.2
Maximum drawdown	30 %	26 %
Average drawdown	6 %	9 %
Number of trades	17	456
Average profit per trade	10.96 % / \$2,192	0.47 % / \$55.91
Average trade duration	20 days	27 hours

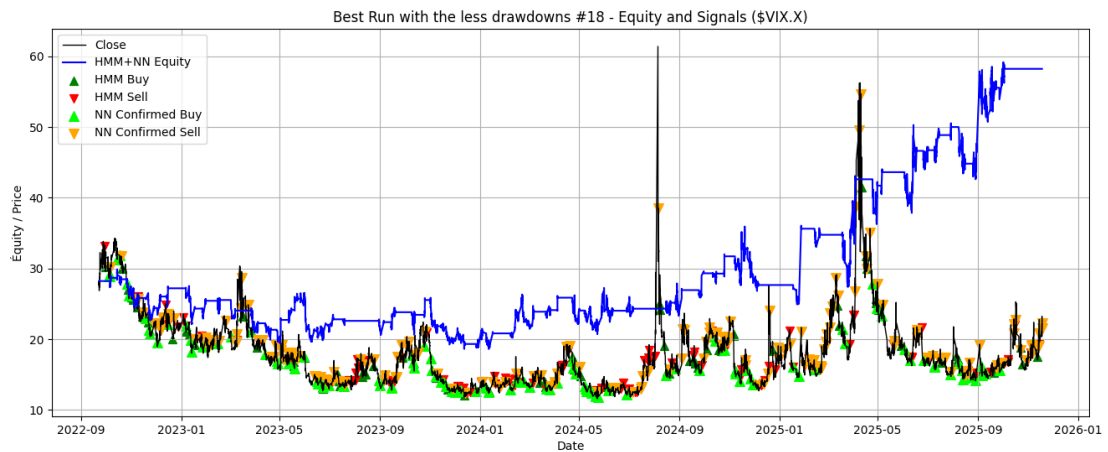


Figure 28: Backtest in 5 min timeframe

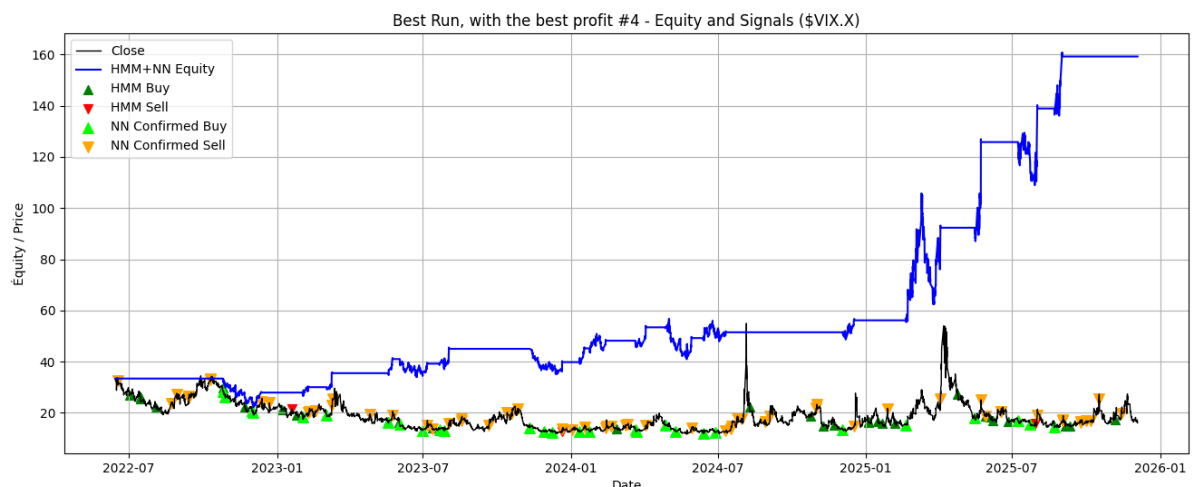


Figure 29: Backtest in 2h timeframe

18 Strategy Performance Using Only Hidden Markov Model

	2h timeframe trading	5 min timeframe trading
Duration of the backtest	3 years	3 years
Total return	247 %	345 %
Annual return	38.85 % / \$11,370	53.47 % / \$9,520
Annual volatility	19 %	9 %
Sharpe ratio	2.39	1.46
Maximum drawdown	32 %	42 %
Average drawdown	7 %	19 %
Number of trades	70	924
Average profit per trade	9.25 % / \$1,912	0.18 % / \$36
Average trade duration	13 days	20 hours

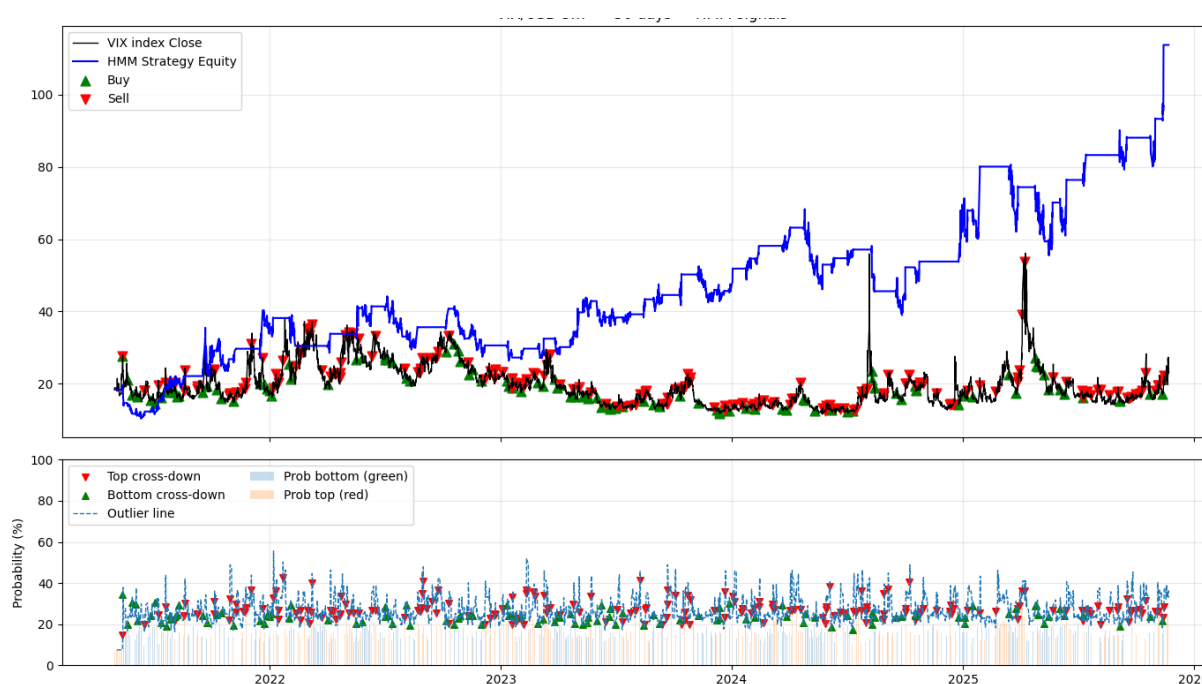


Figure 30: Bayesian HMM Results

19 Real-Time Work

19.1 TradeStation API

Once we obtained a robust model, we then needed to test it in real-time to verify that performance was similar to the simulation.

To achieve this, we had to work with the brokerage platform TradeStation, and use the associated [API](#) with a professional account allowing access to the market and positioning upon it.

To access data in real-time, we only had one account for 2 people; therefore, we had to create an architecture allowing multiple people to use the [API](#). We created a Docker environment to easily deploy the API on different servers. Docker was used to avoid version compatibility issues between different Linux environments.

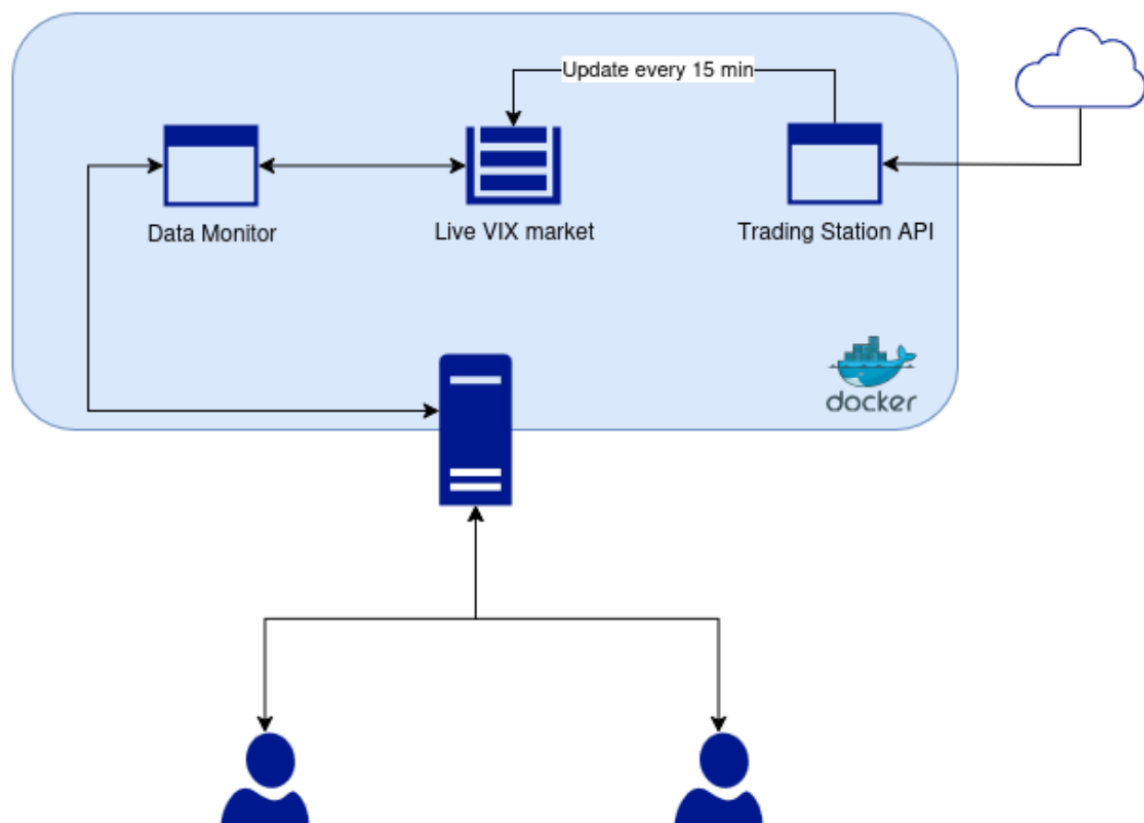


Figure 31: Server environment

As illustrated in the figure above, when active, the service updates its database via the TradeStation API and automatically synchronizes its clock with TradeStation servers to guarantee a real-time signal. Furthermore, the server database manages different clients using SQL queries to avoid conflicts when reading data. The server also integrates an HTTP monitor to verify its proper functioning.

After deploying the [API](#) on the server, a secure connection between the different laboratory computers had to be established. Thus, another team member developed a second client-side service to ensure fast and stable synchronization with the server. Moreover, to simplify the integration of the continuous stream signal into our Python scripts, a dedicated library (illustrated on the right) was designed, allowing any Python program to simultaneously access real-time data. This is achieved through background threads that automatically connect to the client service and continuously transmit the incoming market signal to the agent.

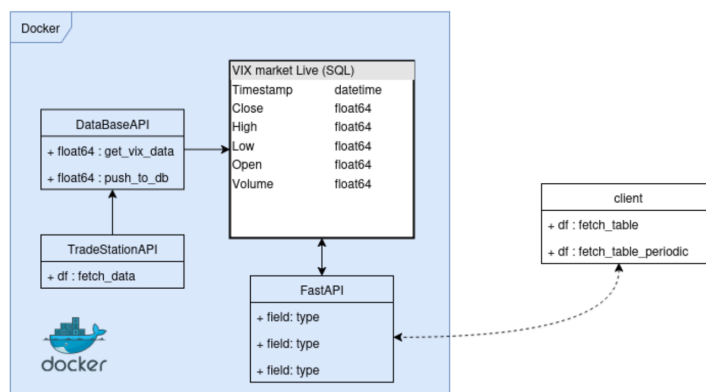


Figure 32: Server/client interface

19.2 Real-Time Implementation

The primary objective of real-time implementation is to verify that order execution occurs exactly as anticipated during simulation. In a live market, discrepancies often arise compared to simulations when purchasing an asset. For instance, if one intends to buy a product priced at \$100, it might only be available at \$100.3 in reality, resulting in an immediate loss. This phenomenon is known as [slippage](#).

While this is just one example, real-time performance must remain consistent with simulations. Furthermore, our model must be capable of reacting to various unexpected simple errors that may occur, such as network interruptions, program crashes, or market scenarios not anticipated by the strategy.

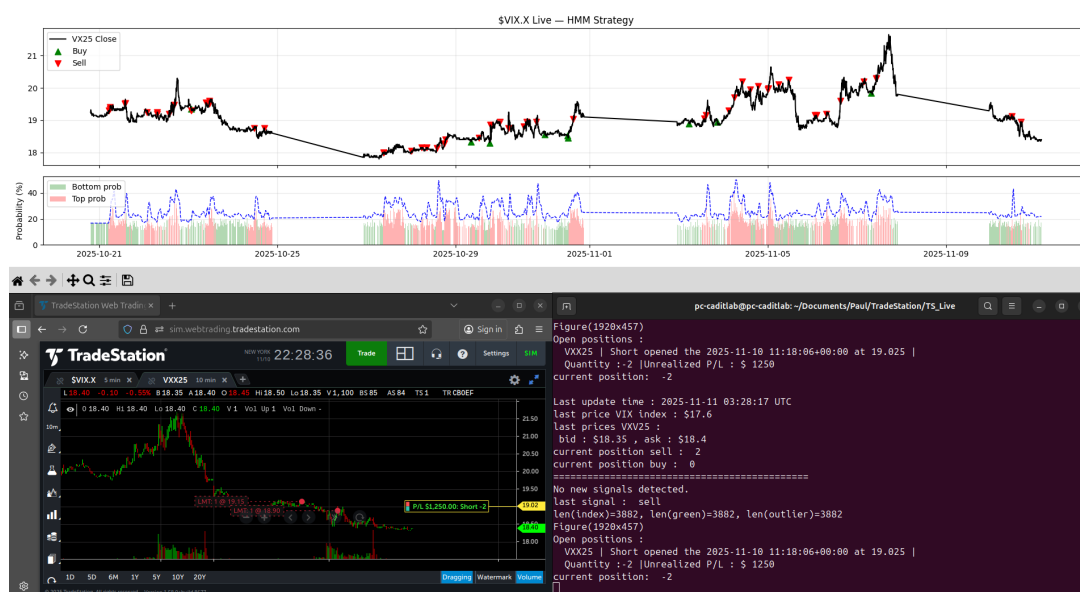


Figure 33: Real-time interface

20 Future Work

The developed model performs well and meets the requirement of being able to filter the market effectively. Additionally, the integration with neural networks slightly improves its efficiency.

Parallel to my work, another team member developed a Reinforcement Learning tool—a [PPO](#) with an actor and agent architecture. It would be highly relevant for future work to integrate the Hidden Markov Model as an actor within this infrastructure.

Furthermore, to improve speed and robustness in real-time execution, it would have been pertinent to develop the core code in C++ to minimize latency.

21 Conclusion

During this internship, I had the opportunity to apply complex mathematics in detail and study how mathematical principles can be applied to finance and, more broadly, to Deep Learning.

This experience allowed me to discover a field that is not necessarily common for an electrical engineering student: market finance. I was already curious about the financial world before starting this internship, and this experience allowed me to dive into it and analyze it in depth. During this semester, I also completed my PPH on the question: "The world of finance: a market governed by the most powerful?". I was able to discuss this with professionals who shed light on the matter.

I was also able to apply skills acquired in numerical analysis, signal processing, and mathematics courses. Creating a model from scratch requires a deep understanding of the tools used, which helped me distinguish aspects of Machine Learning that I had not previously considered. Working with experienced individuals on these complex subjects is extremely rewarding.

Finally, this internship was conducted in Seoul, South Korea, and it was a powerful experience for me. Discovering the Korean working world—which is very different from that of France—allowed me to open my horizons. I also discovered Korean culture, which provided a total change of scenery.

This experience allowed me to improve my English proficiency, which is a key asset in today's professional world.

Acronyms

API Application Programming Interface

AI Artificial Intelligence

BCE Binary Cross Entropy

FPGA Field Programmable Gate Array

HMM Hidden Markov Model

LLM Large Language Model

LSTM Long Short Term Memory

MSE Mean Squared Error

MLP Multi-Layer Perceptron

PPO Proximal Policy Optimization

SLAM Simultaneous Localization And Mapping

SNU Seoul National University

Definitions of Terms

Futures Futures contracts are financial instruments that allow market participants to offset or assume the risk of price changes for an asset over time.. [10](#), [22](#), [36](#)

sharpe ratio The Sharpe ratio indicates whether portfolio returns are attributable to wise decisions or luck. It compares the return of an investment to its risk. A higher Sharpe ratio signifies a more robust strategy.. [24](#), [36](#)

slippage Slippage refers to the difference between the expected price of a trade and the price at which it is executed. Slippage can occur at any time but is more frequent during periods of high volatility.. [33](#), [36](#)

22 References

References

- [1] Monteiro, Tiago. 2024. *AI-Powered Energy Algorithmic Trading: Integrating Hidden Markov Models with Neural Networks*. arXiv preprint arXiv:2407.19858v5.
- [2] Nguyen, Nguyet, and Dung Nguyen. 2018. *Global Stock Selection with Hidden Markov Model*. Youngstown, OH: Department of Mathematics and Statistics, Youngstown State University; Venice, FL: Ned Davis Research Group.
- [3] Oelschläger, Lennart, Timo Adam, and Rouven Michels. 2020. *fHMM: Hidden Markov Models for Financial Time Series in R*. Bielefeld: Bielefeld University; St Andrews: University of St Andrews.
- [4] Song, Xingyao, Wenyu Chen, and Junyi Lu. 2021. *Research on Financial Stock Market Prediction Based on the Hidden Quantum Markov Model*. Sydney: Business School, The University of Sydney; Chengdu: University of Electronic Science and Technology of China.
- [5] Yuan, Yuan. 2019. *Market Regime Identification Using Hidden Markov Model*. London: Department of Computer Science, University College London.